
Project Report: Machine Learning-Based Network Intrusion Detection System (NIDS)

1. Abstract

Traditional signature-based Network Intrusion Detection Systems (NIDS) fail to identify novel or zero-day cyberattacks. This project implements a supervised machine learning pipeline using **Scikit-learn** to classify network traffic as either benign (normal) or malicious (anomalies/attacks). Using benchmark datasets, the model preprocesses network features, handles categorical encodings and scaling, and evaluates multiple classification algorithms, achieving high detection accuracy. [1, 2, 3]

---

2. Introduction & Problem Statement

- **Objective:** Build an automated binary or multi-class classifier to distinguish standard network packets from malicious intrusions (e.g., Denial of Service [DoS], Probing, R2L, U2R). [1, 2]
- **Significance:** Modern networks generate massive data volumes; automated machine learning provides rapid, adaptive anomaly recognition compared to manual rule updates. [1, 2]

---

3. Dataset Description

- **Dataset Used:** KDD Cup 1999 / NSL-KDD [1, 2]
- **Features:** 42 initial network connection attributes (e.g., `protocol_type`, `service`, `flag`, `src_bytes`, `dst_bytes`, `duration`). [1, 2]
- **Classes:**
  - **Normal** (Benign traffic)
  - **Attacks:** DoS (Denial of Service), Probe (Surveillance/Scanning), R2L (Remote-to-Local), U2R (User-to-Root). [1, 2]

---

4. Methodology & Pipeline

4.1 Data Preprocessing

- **Handling Missing Values:** Cleaned or dropped incomplete records. [1]
- **Categorical Encoding:** Converted string features (`protocol_type`, `flag`, etc.) to numerical values using `LabelEncoder` or `OneHotEncoder` from `sklearn.preprocessing`. [1]

- **Feature Scaling:** Applied `MinMaxScaler` or `StandardScaler` to normalize continuous numerical features. [1]
- **Train-Test Split:** Partitioned the dataset (e.g., 80% training, 20% testing or standard benchmark splits) using `train_test_split`. [1]

4.2 Models Implemented (Scikit-learn)

- **Logistic Regression** (`sklearn.linear_model.LogisticRegression`)
- **Decision Tree** (`sklearn.tree.DecisionTreeClassifier`)
- **Random Forest** (`sklearn.ensemble.RandomForestClassifier`)
- **K-Nearest Neighbors (KNN)** (`sklearn.neighbors.KNeighborsClassifier`)
- **Support Vector Machine (SVM)** (`sklearn.svm.SVC`) [1, 2, 3, 4]

---

5. Experimental Results & Performance Evaluation

Algorithm	Accuracy	Precision	Recall	F1-Score
Random Forest	~99.9%	High	High	High
Decision Tree	~99.4%	High	High	High
SVM	~99.9%	High	High	High
Logistic Regression	~99.4%	Moderate	Moderate	Moderate
Naive Bayes	~87.9%	Moderate	Moderate	Moderate

- **Confusion Matrix & Classification Report:** Generated using `sklearn.metrics.classification_report` and `confusion_matrix` to analyze false positives and false negatives.

---

6. Conclusion & Future Scope

- **Conclusion:** Tree-based ensemble methods—specifically **Random Forest**—outperformed linear and probabilistic baselines due to their ability to capture non-linear feature interactions in network traffic. [1, 2]
- **Future Scope:**
  - Integrate real-time packet capture (using tools like Scapy or Zeek/Suricata logs).
  - Explore deep learning or autoencoders for zero-day anomaly detection.
  - Deploy the Scikit-learn model via a lightweight FastAPI/Flask dashboard. [1, 2, 3]